

Complete CI/CD Pipeline Setup

Spring Boot on Azure Kubernetes Service

A Step-by-Step Guide

Using Jenkins, Docker, ACR, and Azure CLI

August 31, 2026

Contents

1	Introduction	2
2	Prerequisites and Environment Setup	3
3	Azure Resource Creation	4
3.1	Resource Group and Providers	4
3.2	AKS Cluster Creation	4
3.3	Azure Container Registry (ACR)	4
4	Jenkins Configuration	5
4.1	Plugin Installation	5
4.2	Global Tool Configuration	5
4.3	Credentials Setup	5
5	Application Preparation	6
5.1	Kubernetes Manifest	6
5.2	Application Configuration	6
6	Jenkins Pipeline Implementation	7
6.1	Create Pipeline Job	7
6.2	Jenkinsfile (Pipeline Script)	7
6.3	Run the Pipeline	8
7	Deployment Verification	10
8	Clean Up Resources	11
9	Quick Command Reference	12
10	Conclusion	13

1 Introduction

This document provides a comprehensive, step-by-step guide to setting up a complete CI/CD pipeline for deploying a Spring Boot application to Azure Kubernetes Service (AKS). The pipeline utilizes:

- **Jenkins** for automation and orchestration
- **Docker** for containerization
- **Azure Container Registry (ACR)** for image storage
- **Azure Kubernetes Service (AKS)** for container orchestration

2 Prerequisites and Environment Setup

Table 1: Prerequisite Setup Steps

Step	Action	Command
2.1	Install Azure CLI	Download from: https://docs.microsoft.com/cli/azure/install-azure-cli
2.2	Login to Azure	<code>az login</code>
2.3	Set Active Subscription	<code>az account set -s <your-subscription-id></code>
2.4	Install Docker Desktop	Download from: https://www.docker.com/products/docker-desktop
2.5	Test Docker Installation	<code>docker run hello-world</code>
2.6	Install kubectl	<code>az aks install-cli</code>
2.7	Verify kubectl Version	<code>kubectl version --client</code>

3 Azure Resource Creation

3.1 Resource Group and Providers

Table 2: Resource Group Setup

Step	Action	Command
3.1.1	Create Resource Group	<code>az group create --name rg-aks --location eastus</code>
3.1.2	Register AKS Provider	<code>az provider register --namespace Microsoft.ContainerService</code>
3.1.3	Register ACR Provider	<code>az provider register --namespace Microsoft.ContainerRegistry</code>

3.2 AKS Cluster Creation

Table 3: AKS Cluster Creation

Step	Action	Command
3.2.1	Create AKS Cluster	<code>az aks create --resource-group rg-aks --name aks-cluster-01 --node-count 1 --generate-ssh-keys --node-vm-size Standard_D2s_v7</code>
3.2.2	Get Cluster Credentials	<code>az aks get-credentials --resource-group rg-aks --name aks-cluster-01</code>

3.3 Azure Container Registry (ACR)

Table 4: ACR Setup

Step	Action	Command
3.3.1	Create ACR Registry	<code>az acr create --resource-group rg-aks --name rgacr2802 --sku Basic --admin-enabled true</code>
3.3.2	Get ACR Credentials	<code>az acr credential show --name rgacr2802</code>
3.3.3	Attach ACR to AKS	<code>az aks update -n aks-cluster-01 -g rg-aks --attach-acr rgacr2802</code>

4 Jenkins Configuration

4.1 Plugin Installation

From **Manage Jenkins** $\rightarrow$ **Plugins**, install these required plugins:

- Docker
- Docker Pipeline
- Kubernetes CLI
- Kubernetes Credentials
- Git
- Maven Integration
- Pipeline

4.2 Global Tool Configuration

From **Manage Jenkins** $\rightarrow$ **Tools**:

- Add JDK installation (e.g., JDK 17)
- Add Maven installation

4.3 Credentials Setup

From **Manage Jenkins** $\rightarrow$ **Credentials** $\rightarrow$ **System** $\rightarrow$ **Global credentials**:

Table 5: Jenkins Credentials

Credential	Type	ID	Details
ACR Credentials	Username with password	ACR-Credential	Username: rgacr2802 Password: (from <code>az acr credential show</code>)
AKS Credentials	Kubernetes configuration	AKS-Kubeconfig	Paste content of <code>~/.kube/config</code>

5 Application Preparation

5.1 Kubernetes Manifest

Create `jenkins-aks-deploy-from-acr.yaml` in your repository root:

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: my-springboot-deployment
5 spec:
6   replicas: 3
7   selector:
8     matchLabels:
9       app: springboot-app
10  template:
11    metadata:
12      labels:
13        app: springboot-app
14    spec:
15      containers:
16        - name: my-springboot-app
17          image: rgacr2802.azurecr.io/springboot-app:latest
18          imagePullPolicy: Always
19          ports:
20            - containerPort: 8085
21  ---
22  apiVersion: v1
23  kind: Service
24  metadata:
25    labels:
26      app: springboot-app
27    name: springboot-app
28  spec:
29    ports:
30      - name: http
31        port: 80
32        protocol: TCP
33        targetPort: 8085
34    type: LoadBalancer
35    selector:
36      app: springboot-app
```

5.2 Application Configuration

Ensure your `application.properties` contains:

```
1 server.port=8085
```

6 Jenkins Pipeline Implementation

6.1 Create Pipeline Job

1. In Jenkins, click **New Item**
2. Enter name (e.g., `springboot-aks-deploy`)
3. Select **Pipeline** and click **OK**

6.2 Jenkinsfile (Pipeline Script)

Replace the pipeline script with the following:

```
1 pipeline {
2     agent any
3
4     environment {
5         registryName = "rgacr2802"
6         registryUrl = "rgacr2802.azurecr.io"
7         registryCredential = 'ACR-Credential'
8         kubeConfigCredential = 'AKS-Kubeconfig'
9         imageTag = "${env.BUILD_NUMBER}"
10        appName = "springboot-app"
11    }
12
13    stages {
14        stage('Checkout') {
15            steps {
16                git branch: 'main', url: 'https://github.com/
17                    ravinikampm/springboot-app.git'
18            }
19        }
20
21        stage('Build with Maven') {
22            steps {
23                bat 'mvn clean install'
24            }
25        }
26
27        stage('Build Docker Image') {
28            steps {
29                script {
30                    bat "docker build --load -t ${registryUrl}/${
31                        appName}:${imageTag} ."
32                }
33            }
34        }
35
36        stage('Push to ACR') {
37            steps {
38                script {
```

```

37         docker.withRegistry("https://${registryUrl}",
38             registryCredential) {
39             bat "docker push ${registryUrl}/${appName}
40                 :${imageTag}"
41         }
42     }
43
44     stage('Deploy to AKS') {
45         steps {
46             script {
47                 withKubeConfig([credentialsId:
48                     kubeConfigCredential]) {
49                     bat "kubectl apply -f jenkins-aks-deploy-
50                         from-acr.yaml"
51                     bat "kubectl set image deployment/my-
52                         springboot-deployment my-springboot-
53                         app=${registryUrl}/${appName}:${imageTag}"
54                     bat "kubectl rollout status deployment/my-
55                         springboot-deployment --timeout=5m"
56
57                     def podStatus = bat(script: "kubectl get
58                         pods -l app=springboot-app -o jsonpath
59                         ='{.items[*].status.phase}'",
60                         returnStdout: true).trim()
61                     if (!podStatus.contains("Running")) {
62                         error "Deployment failed: Not all
63                             pods are running"
64                     }
65
66                     def serviceIp = bat(script: "kubectl get
67                         service springboot-app -o jsonpath='{
68                             .status.loadBalancer.ingress[0].ip}'",
69                         returnStdout: true).trim()
70                     echo "Application accessible at: http://$
71                         {serviceIp}"
72                 }
73             }
74         }
75     }
76 }

```

6.3 Run the Pipeline

1. Click **Save**
2. Click **Build Now**

3. Monitor the console output for each stage

7 Deployment Verification

Table 6: Verification Commands

Action	Command
Verify Pods	<code>kubectl get pods</code>
Verify Services	<code>kubectl get services</code>
Access Application	Open browser at <code>http://<EXTERNAL-IP></code>
View Rolling Update History	<code>kubectl rollout history deployment/my-springboot-deployment</code>

8 Clean Up Resources

To avoid ongoing Azure costs when no longer needed:

```
1 az group delete --name rg-aks --yes --no-wait
```

9 Quick Command Reference

Table 7: Quick Command Reference

Action	Command
Login to Azure	<code>az login</code>
Create Resource Group	<code>az group create --name rg-aks --location eastus</code>
Create AKS Cluster	<code>az aks create -g rg-aks -n aks-cluster-01 --node-count 1 --generate-ssh-keys --node-vm-size Standard_D2s_v7</code>
Get AKS Credentials	<code>az aks get-credentials -g rg-aks -n aks-cluster-01</code>
Create ACR	<code>az acr create -g rg-aks -n rgacr2802 --sku Basic --admin-enabled true</code>
Get ACR Credentials	<code>az acr credential show --name rgacr2802</code>
Attach ACR to AKS	<code>az aks update -n aks-cluster-01 -g rg-aks --attach-acr rgacr2802</code>
Test Docker	<code>docker run hello-world</code>
View Pods	<code>kubectl get pods</code>
View Services	<code>kubectl get services</code>
View Docker Images	<code>docker images</code>
Delete Resource Group	<code>az group delete --name rg-aks --yes --no-wait</code>

10 Conclusion

This document captures the entire process of setting up a production-ready CI/CD pipeline for deploying Spring Boot applications to AKS using Jenkins, Docker, and Azure Container Registry. The pipeline ensures:

- Automated builds and containerization
- Zero-downtime rolling updates
- Built-in verification steps
- Full traceability with build-number based image tagging